Supported Expression Elements

See a list of the supported elements that you can use to build dynamic expressions.

> **Tip**
> For practical examples on how to perform various operations by using the supported constructs and functions, see the linked documentation in each category.

Mathematical Operations

For examples and more information, see Mathematical Operations Examples.

Element	Type	Description / Example
Standard operations: Addition, subtraction, multiplication, integer division, modulo division	Operation	Example: `.number + 1, .number – 1, .number * 10, .number / 10, .number % 10`
`floor`	Function	Rounds a number down to the nearest integer
`ceil`	Function	Rounds a number up to the nearest integer
`round`	Function	Rounds a number to the nearest integer

String Manipulation

For examples and more information, see String Manipulation Examples.

Element	Type	Description / Example
String merging	Operation	Example: `"Hi, my name is " + "John"`
String interpolation	Construct	Example:

		`"Hello \(.name)"`
String slices	Construct	Example: `"caterpillar"[5:]`
`toUpperCase`	Function	Converts string to uppercase
`toLowerCase`	Function	Convert string to lowercase
`strip`	Function	Removes the whitespaces around a string
`stripStart`	Function	Removes the whitespaces before a string
`stripEnd`	Function	Removes the whitespaces after a string
`removePrefix`	Function	Trim a string by removing the prefix
`ltrimstr`	Function	The same as `removePrefix`
`removeSuffix`	Function	Trim a string by removing the suffix
`rtrimstr`	Function	The same as `removeSuffix`
`sub("hello"; "world")`	Regular expression	Replaces only the first occurrence of "hello" with "world"
`gsub("hello"; "world")`	Regular expression	Replaces all non-overlapping occurrences of "hello" with "world"

Data Selection and Filtering

For examples and more information, see Data Selection and Filtering Examples.

Element	Type	Description / Example
Current data / identity: `.`	Construct	Takes an input and produces the same value as output. See Identity. You can use it to access object values.
`global data($)`	Construct	Because the identity `(.)` is limited to the scope defined by a pipe `(|)`, Automation Pilot provides a way of accessing the execution's data, regardless of what the current scope is – `global data ($).` `$` acts the same way as `.` but returns the data from the global scope instead. It can be combined with selectors and value iterators (for example, `$.value`) and can also be a part of operations (for example, `.value == $.expectedValue`). If no pipes are involved in an expression, `$` and `.` will return the same

		values. The same restrictions are applicable for both. For more information, see General Considerations and Restrictions. **Example**
Data selectors	Construct	Example: `.object.foo, .object."foo", .object["foo"], .array[0], .object[.inner.filter], "caterpillar"[3]`
Value iterators	Construct	Example: `.array[ ], .object[ ]`
Array slices	Construct	Example: `.array[0:3], .array[0:-1], .array[1:]`
`getCaseInsensitive`	Function	Checks whether a plain (or a stringified) object contains a key and returns the corresponding value in a case-agnostic manner
`filter`	Function	Filters the elements of an array or map based on a condition
`select`	Function	Filters the elements of an array or map based on a condition

Data Manipulation

For examples and more information, see Data Manipulation Examples.

Element	Type	Description / Example
Merging data in arrays and objects	Operation	Example: `$(["John", "Bob"] + ["James", "Monica"])` → `["John", "Bob", "James", "Monica"]`
Replacing data in arrays and objects	Operation	Example: `["John", "Bob"]` → `$(map(if . == "John" then "James" else . end))` → `["James", "Bob"]`
`add`	Function	Adds together (sums, merges, or concatenates) the elements of an array
`map`	Function	Applies a function to all elements of the

		input and returns an array
`sort`	Function	Sorts the array elements in an ascending order
`sortBy`	Function	Sorts the array elements in an ascending order, where each element is transformed with the given expression before it is compared to other elements
`reverse`	Function	Sorts the array in reverse order
`unique`	Function	Sorts the array elements in an ascending order, and removes all duplicate elements by leaving only the first occurrence
`uniqueBy`	Function	Sorts the array elements in an ascending order, and removes all duplicate elements. Each element is transformed with the given expression before it is compared to other elements.
`reduce`	Function	Processes each value from a given stream, applying an expression to combine them into a single accumulated result.

Data Analysis

For examples and more information, see Data Analysis Examples.

Element	Type	Description / Example
Number comparison	Operation	Example: `.number > 1, .number >= 1, .number < 1, .number <= 1, .number == 1, .number != 1`
Boolean operations	Operation	Example: `.condition1 and .condition2, .condition1 or .condition2`
`length`	Function	Returns the length of an object or an array
`keys`	Function	Returns an array of all keys in an object
`values`	Function	Returns an array of all values in an object
`contains`	Function	Checks if the argument value is

		completely contained within the input value
`has`	Function	Checks if an object has a given key, or if an array has an element at this index
`in`	Function	Checks whether or not the input is a key in the given object, or if it is a valid index of an element in the given array
`valueIn`	Function	Checks if the input is a value in an object or an array
`inside`	Function	Checks if the input array is completely contained within the argument array. It works in the opposite way of the `contains` function.
`any`	Function	Checks if at least a single element of an array satisfies a given condition
`all`	Function	Checks if all elements of an array satisfy a given condition
`startsWith`	Function	Checks if the input starts with the given string argument
`endsWith`	Function	Check if the input ends with the given string argument

Data Type Conversion

For examples and more information, see Data Type Conversion Examples.

Element	Type	Description / Example
`toObject`	Function	Converts the value to an object
`toMap`	Function	Creates an object from an array by using one mapping expression for the key and another for the value
`toArray`	Function	Converts the value to an array
`fromEntries`	Function	Transforms an array of objects into a single object
`toEntries`	Function	Converts an object into an array of objects
`toString`	Function	Converts the value to a string
`toEscapedJson`	Function	Stringifies the input and escapes all JSON special characters

`join`	Function	Concatenates array elements into a string with a specified separator
`split`	Function	Splits a string into an array of substrings
`toBase64`	Function	Converts the input value to text and applies Base64 encoding
`fromBase64`	Function	Expects a Base64-encoded text and tries to decode it
`toUrlEncoded`	Function	Converts the value to text and applies URL encoding
`fromUrlEncoded`	Function	Decodes text from URL-encoded format
`toPercentEncoded`	Function	Applies URL encoding and replaces unsafe characters
`toMd5`	Function	Calculates the MD5 hash of a value
`toSha256`	Function	Calculates the SHA-256 hash of a value
`fromYaml`	Function	Converts a provided YAML string into a JSON value
`toYaml`	Function	Converts the provided input to a YAML string
`fromProperties`	Function	Creates a JSON object from a properties-formatted string
`toProperties`	Function	Create a properties-formatted string from a JSON
`fromXml`	Function	Converts a provided XML string into a JSON object
`toNumber`	Function	Converts the value to a number
`toBoolean`	Function	Converts the value to a boolean

Utility Operations

For examples and more information, see Utility Operations Examples.

Element	Type	Description / Example
`if` statement	Construct	If-then-else condition
Variables	Construct	jq lets you define variables using expression as `$variable | expression`. The expression `exp as`

		`$x \| ...` means: for each value of expression exp, run the rest of the pipeline with the entire original input, and with `$x` set to that value.
Alternative operation `//`	Operation	The // operator outputs all the values from its left-hand side excluding those that are false or null. If the left-hand side doesn't yield any values other than false or null, then // returns all the values from its right-hand side. See Alternative operator.
Pipe operation `|`	Operation	The | operator connects two filters by passing the output of the left filter directly into the input of the right filter. See Pipe.
Comma operation	Operation	Example: `1, 2, 3, [1, 2, 3], .number * 1, 2`
`range(N)`	Function	Produces zero or more integer numbers starting from 0 and going up to N (exclusive)
`now`	Function	Returns the current date and time as a timestamp
`nowMillis`	Function	Returns the current date and time in milliseconds
`toDate`	Function	Converts a timestamp (in milliseconds) to a desired date-time string
`fromDate`	Function	Converts a date-time string to a timestamp (in milliseconds)
`guid`	Function	Generates a random GUID
`guidShort`	Function	Generates a random GUID without the hyphens
`isGuid`	Function	Checks if a given text value is a GUID
`compareVersions`	Function	Takes two standard versions either as an object or as an array and returns a number
`compareUnixVersions`	Function	Takes two standard versions either as an object or as an array, compares them using the Unix version sort algorithm, and returns a number
`toPrettyJsonString`	Function	Converts a JSON object into a formatted human-readable JSON string

`toMinifiedJsonString`	Function	Converts a formatted JSON object into a compact, single-line JSON string

Related Information

jq 1.6 Manual